

Name of the Student	KOTHAKOTA RAKESH
Reg. No	192525075
GitHub Link	https://github.com/Rakesh192525075/MLA0403.git

SIMATS ENGINEERING

CO2 - ASSESSMENT TOOL 3

Class Test

COURSE NAME: Deep Learning
Faculty : Dr. Arul Raj A.M

COURSE CODE: MLA04
Slot :

COURSE OUTCOME COVERED

CO 2: Analyze Machine Learning and Deep Learning models by evaluating neural network architectures, representation learning, activation functions, unsupervised learning techniques, and their real-world applications. (L4)

CO Weightage : 15%

Assessment Tool Weightage: 20%

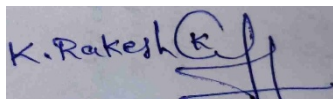
Duration: 90 minutes

Marks: 25

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student

Name of the student: K Rakesh

Class Test

CO2-AT3

QUESTIONS: 05

Total Marks:25

1. Differentiate between Machine Learning and Deep Learning with respect to architecture, data requirements, feature extraction and applications. Give two real-world applications of each.

1. Difference Between Machine Learning and Deep Learning		
Aspect	Machine Learning (ML)	Deep Learning (DL)
Architecture	Usually uses simpler models such as Decision Trees, SVM, KNN, and Linear Regression.	Uses multi-layer neural networks with many hidden layers.
Data Requirements	Can work well with relatively smaller datasets.	Generally requires large amounts of data to achieve good performance.
Feature Extraction	Features usually need to be selected or engineered manually.	Automatically learns features from raw data through multiple neural network layers.
Training Time	Generally faster and requires less computational power.	Usually requires more training time and powerful GPUs/TPUs.
Complexity	Suitable for simpler patterns and structured/tabular data.	Suitable for highly complex patterns such as images, audio, and natural language.
Interpretability	Many ML models are relatively easier to interpret.	Deep neural networks are generally more difficult to interpret.
Applications	Fraud detection, spam filtering, credit scoring, etc.	Image recognition, speech recognition, autonomous driving, etc.

Feature Extraction Difference

Machine Learning:

Raw Data → Manual Feature Extraction → ML Algorithm → Prediction

Deep Learning:

Raw Data → Neural Network → Automatic Feature Learning → Prediction

Real-World Applications

Machine Learning:

1. **Credit-card fraud detection** – ML models identify unusual transaction patterns.
2. **Email spam detection** – ML classifies emails as spam or legitimate.

Deep Learning:

1. **Face recognition** – Deep neural networks learn complex facial features and identify people.
2. **Speech recognition** – Deep learning models convert spoken language into text.

Conclusion

Machine Learning generally depends more on **human-designed features** and can work effectively with smaller datasets. **Deep Learning** uses **multiple neural-network layers to automatically learn features** and is particularly powerful for large, complex datasets such as images, speech, and text.

2. Explain the concept of **Representation Learning**. Discuss how the **width** and **depth** of a neural network influence its learning capability, computational complexity and model performance.

1. Representation Learning

Representation Learning is a technique in which a neural network **automatically learns useful features or representations from raw data**, instead of requiring humans to manually select features.

For example, in **image recognition**, the network can automatically learn:

Raw Image

↓

Early Layers → Edges and Lines

↓

Middle Layers → Shapes and Patterns



Deep Layers → Eyes, Nose, Face Features



Output → Person/Class

Thus, representation learning transforms raw data into a **useful internal representation** that makes prediction or classification easier.

Examples:

- Images → edges → shapes → objects
- Speech → frequencies → sounds → words
- Customer data → purchase patterns → customer groups

2. Width of a Neural Network

Width refers to the **number of neurons in a layer**.

Example:

Narrow Network: 4 → 4 → 4 neurons

Wide Network: 64 → 128 → 64 neurons

Effect of increasing width

- **Learning capability:** More neurons can learn more features and patterns.
- **Computational complexity:** More neurons increase the number of parameters and calculations.
- **Performance:** Increasing width can improve performance, especially when the original network is too small. However, excessive width can increase memory usage and may lead to overfitting.

3. Depth of a Neural Network

Depth refers to the **number of layers** in a neural network.

Shallow Network:

Input → Hidden → Output

Deep Network:

Input → Hidden 1 → Hidden 2 → Hidden 3 → Hidden 4 → Output

Effect of increasing depth

- **Learning capability:** Deeper networks can learn hierarchical and highly complex representations.
- **Computational complexity:** More layers require more computations and parameters, increasing training time.
- **Performance:** Appropriate depth can significantly improve performance on complex tasks. However, excessive depth can make training difficult and may cause problems such as vanishing/exploding gradients.

4. Width vs Depth

Factor	Width	Depth
Meaning	Number of neurons per layer	Number of layers
Learning	Learns more features in parallel	Learns hierarchical features
Complexity	Increases parameters and computation	Increases layers, computation and training complexity

Performance	Can improve performance up to a point	Often better for complex hierarchical patterns
Risk	Overfitting with excessive width	Difficult training with excessive depth

Simple Example

Consider facial recognition:

Input Image



Layer 1 → Edges



Layer 2 → Shapes



Layer 3 → Eyes / Nose / Mouth



Layer 4 → Face Pattern



Output → Person Identity

Here, **depth** allows the network to learn increasingly complex features, while **width** determines how many different features can be learned at each layer.

Conclusion

Representation learning allows neural networks to automatically learn useful features from raw data. Increasing **width** provides more neurons to learn features in parallel, while increasing **depth** enables the network to learn increasingly complex hierarchical

representations. Both can improve model performance, but they also increase computational requirements, so the architecture must be selected carefully.

3. Compare **ReLU**, **Leaky ReLU (LReLU)** and **ELU (Exponential Linear Unit)** activation functions. Explain their mathematical behavior, advantages, disadvantages and suitable use cases.

Activation functions introduce **non-linearity** into a neural network, allowing it to learn complex patterns.

Mathematical Behavior

1. ReLU (Rectified Linear Unit)

- For $x \geq 0$: output = x
- For $x < 0$: output = 0

2. Leaky ReLU (LReLU)

where α is a small positive value, commonly 0.01.

- Positive values pass normally.
- Negative values produce a small negative output instead of zero.

3. ELU (Exponential Linear Unit)

- Positive values are linear.
- Negative values approach smoothly.

Comparison Table

Feature	ReLU	Leaky ReLU	ELU
Positive input			
Negative input	0		
Negative gradient	0	Small non-zero	Smooth, non-zero

Main advantage	Simple and fast	Reduces dying ReLU problem	Smooth negative region
Main disadvantage	Dying ReLU	Requires choosing	More computationally expensive
Computation	Very low	Low	Higher due to exponential
Suitable use	General deep networks	Networks where dead neurons are a concern	Deep networks where smoother activations are beneficial

Advantages and Disadvantages

ReLU

Advantages:

- Simple and computationally efficient.
- Helps reduce the vanishing-gradient problem for positive inputs.
- Widely used in deep neural networks.

Disadvantage:

- Can suffer from the **dying ReLU problem**, where neurons may output zero for all inputs and stop learning.

Leaky ReLU

Advantages:

- Keeps a small gradient for negative inputs.
- Reduces the dying ReLU problem.
- Computationally inexpensive.

Disadvantage:

- Requires selection of the negative slope .

ELU

Advantages:

- Provides a smooth negative region.
- Helps avoid zero gradients for negative inputs.
- Negative outputs can help keep activations closer to zero-centered.

Disadvantages:

- More computationally expensive than ReLU and Leaky ReLU.
- Requires choosing the parameter .

Suitable Use Cases

- **ReLU**: Default choice for many CNNs and deep neural networks because of its simplicity and speed.
- **Leaky ReLU**: Useful when **dying ReLU** is observed or when maintaining a small negative gradient is desirable.
- **ELU**: Useful when smoother activation behavior and negative outputs are beneficial, particularly when computational cost is less important.

Conclusion

ReLU is the simplest and fastest option, **Leaky ReLU** improves ReLU by allowing a small negative gradient, while **ELU** provides a smoother negative region. In practice, ReLU is often the first choice, with LReLU or ELU considered when ReLU causes training problems.

4. Explain the concept of **unsupervised learning** in neural networks. Describe the working principle of **Restricted Boltzmann Machines (RBMs)** and **Autoencoders** and compare their objectives and applications.

1. Concept of Unsupervised Learning

Unsupervised learning is a type of machine learning in which a neural network learns patterns and structures from **unlabelled data**. Unlike supervised learning, there is no predefined target output or class label.

For example, a system can receive thousands of customer records without labels and automatically discover groups or hidden patterns.

Unlabelled Data



Neural Network



Learn Hidden Patterns



Useful Representation



Clustering / Reconstruction / Anomaly Detection

Common unsupervised neural-network techniques include **Restricted Boltzmann Machines (RBMs)** and **Autoencoders**.

2. Restricted Boltzmann Machine (RBM)

An **RBM** is a generative neural network that learns the hidden relationships and probability distribution within data.

It has two layers:

- **Visible layer** – represents the input/observed data.
- **Hidden layer** – learns hidden features or patterns.

There are connections between visible and hidden units, but **no connections between units within the same layer**.

Hidden Layer

○ ○ ○ ○

/\ /\ /\ /\

/|V|V|V|

○ ○ ○ ○ ○

Visible Layer

↓

Input

Working Principle of RBM

1. Input data is provided to the **visible layer**.
2. The visible units activate hidden units according to learned weights.
3. The hidden layer captures important patterns in the input.
4. The RBM attempts to reconstruct the original input from the hidden representation.
5. The difference between the original and reconstructed data is used during training to update the weights.
6. After training, the RBM can estimate missing or unseen preferences and generate useful representations.

Example

For a movie recommendation system:

User Movie History

↓

Visible Layer

↓

Hidden Layer

(User Preferences)



Predicted Preferences



Movie Recommendations

3. Autoencoder

An **Autoencoder** is a neural network that learns to reproduce its input at the output while passing the data through a **compressed representation**.

It has three main parts:

1. **Encoder**
2. **Latent representation**
3. **Decoder**

Input



Encoder



Latent Representation



Decoder

↓

Reconstructed Output

Working Principle

1. The original data is given to the encoder.
2. The encoder compresses the input into a smaller **latent representation**.
3. The decoder reconstructs the original input.
4. The network minimizes the **reconstruction error**.
5. The encoder therefore learns important features of the input data.

For example:

100 Features

↓

Encoder

↓

10 Features

↓

Decoder

↓

100 Reconstructed Features

The 10 features represent the important information learned from the original 100 features.

Applications

Autoencoders are commonly used for:

- **Dimensionality reduction**

- **Feature/representation learning**
- **Anomaly detection**
- **Noise removal (denoising)**
- Data compression

4. RBM vs Autoencoder

Aspect	RBM	Autoencoder
Type	Generative neural network	Neural network for representation learning
Structure	Visible + Hidden layers	Encoder + Latent + Decoder
Main objective	Learn probability distribution and hidden dependencies	Reconstruct input accurately
Training	Often uses Contrastive Divergence	Usually minimizes reconstruction loss using backpropagation
Output	Can model/generate data and estimate missing values	Reconstructed version of input
Representation	Learns hidden features	Learns compressed latent features
Main applications	Recommendation systems, generative modeling, feature learning	Dimensionality reduction, anomaly detection, denoising
Complexity	Training can be more difficult	Generally easier to train

5. Key Difference in Objectives

RBM

Goal: Learn the underlying probability distribution and dependencies in the data.

Data → Hidden Patterns → Learn Probability Distribution

Autoencoder

Goal: Learn a compact representation that can reconstruct the input.

Data → Compress → Reconstruct → Minimize Reconstruction Error

Conclusion

Both **RBM**s and **Autoencoders** can learn useful representations from **unlabelled data**, but their objectives differ. An **RBM** focuses on learning the underlying probability distribution and relationships between variables, making it useful for recommendation and generative tasks. An **Autoencoder** focuses on reconstructing its input through a latent representation, making it particularly useful for **dimensionality reduction, feature learning, denoising, and anomaly detection**.

5. Explain the concept of Autoencoder and its uses in Deep learning .

1. What is an Autoencoder?

An **Autoencoder** is a type of **neural network used for unsupervised learning**. It learns to represent input data in a compressed form and then reconstruct the original data.

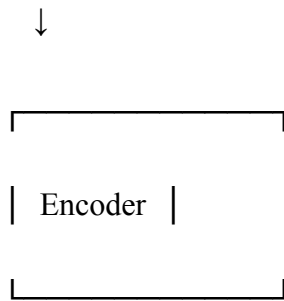
An autoencoder has three main parts:

1. **Encoder** – Converts the input into a smaller representation.
2. **Latent Space** – Stores the important features of the input.
3. **Decoder** – Reconstructs the original input from the latent representation.

Basic Architecture

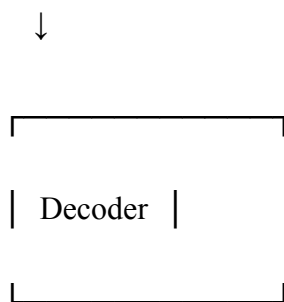
AUTOENCODER

Input Data



↓

Latent Representation



↓

Reconstructed Output

For example:

100 Features → 50 → 10 → 50 → 100 Features

↑

Latent Space

The network tries to make the **reconstructed output as close as possible to the original input**.

2. Working of an Autoencoder

Step 1 – Input:

The original data is given to the network.

Step 2 – Encoding:

The encoder compresses the input and extracts important features.

Step 3 – Latent Representation:

The compressed representation contains the most useful information from the input.

Step 4 – Decoding:

The decoder uses this representation to reconstruct the original data.

Step 5 – Training:

The network minimizes the difference between the original input and reconstructed output.

This difference is called **reconstruction error**.

Original Input



Encoder



Compressed Data



Decoder



Reconstructed Output



Compare with Original



Reconstruction Error



Update Network

3. Uses of Autoencoders in Deep Learning

1. Dimensionality Reduction

Autoencoders can reduce high-dimensional data into a smaller number of features while preserving important information.

Example:

100 customer features → 10 important features.

2. Feature/Representation Learning

The encoder automatically learns useful features from raw data without manually designing them.

3. Anomaly Detection

An autoencoder can learn normal patterns. Abnormal data often produces a **higher reconstruction error**.

Example: Detecting abnormal ECG signals or defective products.

4. Image Denoising

A **denoising autoencoder** can learn to remove noise from images.

Noisy Image → Encoder → Latent Features → Decoder → Clean Image

5. Data Compression

The encoder can compress data into a smaller representation, while the decoder can reconstruct it.

6. Recommendation Systems

Autoencoders can learn user preferences and item relationships and use the learned representations to predict user interests.

4. Advantages

- Does not always require labelled data.
- Automatically learns important features.
- Useful for dimensionality reduction.
- Can detect anomalies.
- Can remove noise from data.
- Useful for learning compact representations.

Conclusion

An **Autoencoder** is an unsupervised neural network that **compresses input data into a latent representation and reconstructs it**. It is widely used in deep learning for **dimensionality reduction, representation learning, anomaly detection, image denoising, data compression, and recommendation systems**.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand